

A Bayesian Information-Gain Agent for CI Failure Diagnosis

Panduru Grishm Siddharth

Independent Researcher

siddharthpg18@gmail.com

Abstract

CI pipeline failures can have several possible root causes, and the available evidence is often insufficient to identify the cause immediately. We developed a CI diagnosis agent that treats the root cause as a hidden state and maintains a probability distribution over possible causes. The V1 agent considers five hidden states: Code, Test, Dependency, CI/Config, and Something else. It begins with empirical prior probabilities obtained from a labeled CI failure dataset and updates its beliefs as diagnostic actions produce new evidence. The diagnostic policy ranks actions using expected information gain relative to diagnostic cost (EIG/cost), allowing the agent to prefer actions that are expected to reduce uncertainty while considering diagnostic effort. A 40-case held-out evaluation compared EIG/cost with a cost-only policy and a historical-probability baseline. EIG/cost achieved 55% accuracy, 65.23% macro precision, 55% macro recall, and 84.6% report precision, while also using fewer actions and lower average diagnostic cost than the cost-only policy. However, the experiment uses simulated action outcomes because the original dataset does not directly contain diagnostic-action outcome probabilities. The results therefore demonstrate the behavior of the proposed decision policy under the constructed simulation rather than production-level diagnostic performance.

1 Introduction

CI/CD pipelines can fail for many different reasons. A failure may be caused by a recent code change, a test or flaky test, a dependency problem, or a CI/environment configuration issue. In some cases, the failure can also be related to infrastructure, an external service, or other anomalous events. The main problem is that the actual cause is not directly observable when the failure first occurs. The agent therefore has to reason about multiple possible causes and decide what to investigate next.

The goal of this project is to build an agent which identifies the cause of a CI pipeline failure and performs diagnostic actions to identify the root cause. Instead of follow-

ing one fixed debugging sequence, the agent uses the context of the failure to decide which diagnostic action should be performed next. This follows the observation from practitioner discussions that engineers generally start by identifying the failed stage and inspecting logs, and then use the available evidence to narrow down the possible causes and next actions. In one discussion, a practitioner described identifying the failed stage as useful for eliminating possible root causes, while another described using the failed stage, logs, code/tests, dependencies, and infrastructure as part of the debugging process. These observations influenced the design of the agent.

The agent represents the possible root causes as hidden states and maintains a belief, represented as a probability, for each state. Historical data is used to obtain the initial prior distribution. When evidence is observed, the agent updates the probabilities using Bayesian reasoning. The agent can then choose a diagnostic action which is expected to provide useful information about the hidden state.

The central decision problem is therefore not only to predict a root cause, but also to decide which action should be performed next. The proposed V1 policy uses expected information gain divided by diagnostic cost to rank available actions. After an action is performed, its outcome is treated as feedback and used to update the posterior distribution. If a root cause reaches a predefined confidence threshold of 90%, the agent reports it. If the available diagnostic actions are exhausted without reaching this threshold, the agent escalates the case for human review.

The quantitative experiment compares three policies: an EIG/cost policy, a cost-only policy, and a baseline based on historical probabilities. The EIG/cost policy achieved 55% accuracy compared with 50% for cost-only and 25% for the baseline.

The main contributions of this work are:

- A CI failure diagnosis formulation in which the root cause is represented as a hidden state spanning five distinct categories with an associated probability.
- A sequential diagnostic policy that ranks actions according to expected information gain relative to diagnostic cost.
- A Bayesian belief update process that uses diagnostic outcomes as feedback.

- An evaluation of EIG/cost against cost-only and historical-probability baseline policies on 40 held-out cases.
- A failure analysis identifying cases where the agent became confidently wrong, was diverted by misleading evidence, or remained below the reporting threshold.

2 Related Work and User Discussions

The initial design of the agent was informed by discussions with developers and practitioners about how CI failures are diagnosed in practice. The purpose of these discussions was not to treat individual responses as ground-truth probability data, but to identify useful evidence, diagnostic actions, and situations where human intervention is appropriate.

A response in a Reddit discussion described first exporting and reading CI logs and then attempting to reproduce the failure. The response also noted that identifying the failed pipeline stage can eliminate several possible root causes. This led to treating the failed pipeline stage as an important input to the agent and to including log inspection as a diagnostic action.

Another practitioner response described a debugging sequence beginning with the failed stage and pipeline logs, followed by code/tests, dependencies, and infrastructure/configuration. Previous incidents and runbooks were also described as useful for confirming a diagnosis. These observations support the idea that diagnosis should be sequential rather than based on a single prediction.

A separate practitioner discussion described a build failure workflow in which the engineer first checks the local build and then investigates dependency, configuration, Docker, hosting, and server state. The practitioner also noted that the exact root cause can sometimes remain unknown even after the failure has been localized. This influenced the decision to allow the agent to escalate rather than forcing it to report a root cause when the available evidence is insufficient.

These discussions also influenced the action-selection policy. Rather than using a fixed action sequence for every failure, the research notes propose selecting actions according to the context of the failure and their expected usefulness or “hit ratio.” The final V1 implementation operationalizes this idea using EIG/cost.

The project also received an architecture review in which the high-level Excalidraw architecture was described as clearly explained. The reviewer suggested consolidating the work into a polished LaTeX paper. This influenced the presentation of the final work but was not used as evidence for the quantitative evaluation.

The discussion record is maintained separately as a project artifact and distinguishes direct responses from background observations.

3 Agent Design

The agent receives a CI failure and attempts to identify its root cause through sequential diagnosis. The V1 architecture consists of an evidence collector, a belief engine, a diagnostic policy, diagnostic actions, feedback, and human escalation.

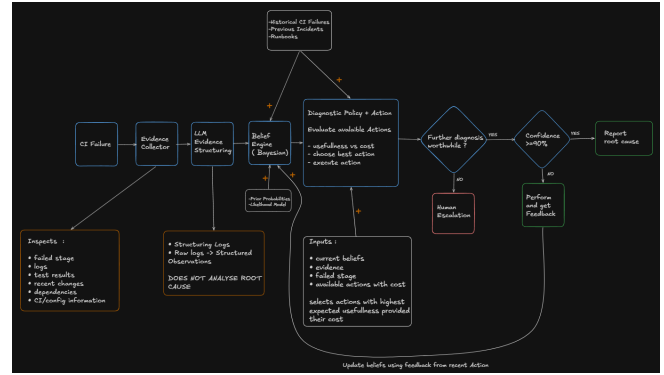


Figure 1: Architecture of the CI Diagnosis Agent. The agent collects evidence, maintains beliefs over hidden root causes, selects diagnostic actions, incorporates action feedback, and escalates to a human when sufficient confidence is not achieved.

The evidence collector provides the context for the failure. The belief engine maintains probabilities over the possible root causes. The diagnostic policy decides which available action should be performed. The result of the action is then returned as feedback to the belief engine, which updates the probabilities. If the agent cannot reach sufficient confidence after useful actions have been exhausted, it escalates the case to a human.

3.1 Agent Input

The primary V1 observable evidence variable is the failed pipeline stage. The dataset did not provide a standardized stage field, so stages were derived from pipeline step names using high-confidence mappings. The resulting stages included Build, Test, Setup/Dependency, Quality/Analysis, Deploy/Publish, Ambiguous, and Unknown.

The research also identifies additional evidence that can be used by a richer version of the agent, including pipeline logs, test results, recent code/test changes, dependency changes, CI/environment configuration changes, differences from the last successful run, local build results, Docker image or cache changes, previous similar incidents, hosting information, and server/session state.

However, the V1 quantitative model does not have direct historical observations for all of these variables. The failed stage therefore acts as the first empirically validated evidence variable.

3.2 Hidden States

The V1 quantitative experiment uses five hidden states:

1. **Code:** the failure is caused by a recent code change or application issue.
2. **Test:** the failure is caused by a test-related issue or flaky test.
3. **Dependency:** the failure is caused by a dependency or dependency-related change.
4. **CI/Config:** the failure is caused by a CI, build, or environment configuration issue.

176 5. **Something else**: the failure is caused by residual, out-
177 of-distribution, infrastructure, or external service issues.

178 3.3 Evidence and Diagnostic Actions

179 The agent can perform several diagnostic actions, including:

- 180 • Inspect pipeline logs
- 181 • Inspect the failed pipeline stage
- 182 • Compare with the last successful run
- 183 • Inspect recent code/test changes
- 184 • Inspect dependency changes
- 185 • Inspect CI/environment configuration
- 186 • Search previous incidents/runbooks
- 187 • Inspect Docker image/cache changes
- 188 • Inspect server/session state

189 The actions are not intended to be executed in one fixed or-
190 der. The failed stage and current beliefs are used to determine
191 which actions are relevant. The policy then ranks the avail-
192 able actions using their expected information gain relative to
193 cost.

194 3.4 LLM Role

195 A future version of the architecture includes an LLM between
196 the evidence collector and the belief engine. The intended
197 role of the LLM is to structure messy or unstructured CI evi-
198 dence into a form that the probabilistic model can process.

199 The LLM is not intended to directly determine the root
200 cause. The research separates evidence interpretation from
201 probabilistic diagnosis: the Bayesian belief engine remains
202 responsible for maintaining the probabilities and the diagnos-
203 tic policy remains responsible for selecting actions.

204 4 Probability Model and Decision Rule

205 The agent represents the unknown root cause as a hidden
206 state. Let

$$H \in \{\text{Code, Test, Dependency, CI/Config, Something else}\}.$$

207 Let E represent the evidence currently available to the agent,
208 a represent a diagnostic action, and o represent an outcome
209 of that action.

210 4.1 Prior Probability

211 Before considering evidence from the current case, the agent
212 starts with an empirical prior estimated from the historical
213 labeled dataset. After applying the V1 hidden-state mapping
214 across all five states, 305 of the 375 records were retained for
215 quantitative probability estimation.

216 The resulting empirical prior was:

217 These probabilities represent the distribution within the in-
218 cluded subset of the dataset and are not treated as universal
219 CI failure probabilities.

220 The prior is conceptually written as:

$$P(H).$$

221 The current failure evidence should not be used to construct
222 the prior. It is instead incorporated through the likelihood
223 model.

Table 1: Empirical prior distribution across the five hidden states ($\epsilon = 0.02$).

Hidden State	Prior Probability
Code	17.35%
Test	38.55%
Dependency	29.24%
CI/Config	12.85%
Something else (ϵ)	2.00%
Total	100.00%

Table 2: Likelihood of observing a Build failure for each hidden state.

Hidden State	$P(\text{Build} H)$
Code	57.4%
Test	35.8%
Dependency	42.9%
CI/Config	27.5%
Something else	10.0%

224 4.2 Likelihood

225 The likelihood describes how likely an observation is under a
226 particular hidden state:

$$P(o | H, E, a).$$

227 For the initial V1 evidence model, the failed pipeline stage
228 is represented as a categorical observable variable. For ex-
229 ample, the empirical likelihoods for observing a Build failure
230 are:

231 The likelihood values are empirical estimates from the la-
232 belated dataset and are not intended to represent universal CI
233 failure statistics.

234 4.3 Bayesian Posterior Update

235 After receiving an observation, the agent updates its belief
236 using Bayes' rule:

$$P(H | E, o, a) = \frac{P(o | H, E, a)P(H | E)}{P(o | E, a)}.$$

237 The denominator normalizes the probabilities:

$$P(o | E, a) = \sum_h P(o | h, E, a)P(h | E).$$

238 Therefore, the posterior is obtained by multiplying the current
239 belief by the likelihood and then normalizing the result.

240 4.4 Entropy

241 The uncertainty in the current belief distribution is repre-
242 sented using entropy:

$$H(H | E) = - \sum_h P(h | E) \log P(h | E).$$

243 A distribution in which the probabilities are spread across
244 several hidden states has greater uncertainty. A distribution
245 in which one state has most of the probability has lower un-
246 certainty.

4.5 Information Gain

For an action outcome o , the information gain is the reduction in entropy caused by observing that outcome:

$$IG(o) = H(H | E) - H(H | E, o, a).$$

Thus, an outcome is useful when it substantially reduces uncertainty about the hidden root cause.

4.6 Expected Information Gain

Before executing an action, the agent does not know which outcome will occur. Therefore, it calculates the expected information gain across the possible outcomes:

$$EIG(a) = \sum_o P(o | E, a) IG(o).$$

The probability of an outcome is obtained by marginalizing over the five possible hidden states:

$$P(o | E, a) = \sum_h P(o | h, E, a) P(h | E).$$

4.7 Action Selection

The diagnostic policy considers both information gain and diagnostic cost. The score of an action is:

$$Score(a) = \frac{EIG(a)}{Cost(a)}.$$

The action with the highest score is selected first.

This means that the policy does not simply choose the action with the highest information gain. An action that provides a large amount of information but requires substantially more diagnostic effort may be ranked below a cheaper action with a better information-to-cost ratio.

4.8 Decision Policy and Stopping Threshold

In the V1 evaluation, the agent stops diagnostic actions and reports root cause H_i as soon as its posterior belief satisfies $P(H_i | E) \geq 0.90$. If the available candidate actions are exhausted without any hidden state reaching this threshold, the agent escalates the case for human review.

5 Test Method

5.1 Dataset

The original replication dataset contains 375 labeled CI failure jobs. It contains root-cause descriptions, sub-categories, broader categories, pipeline steps, and associated actions/scripts.

The V1 agent uses a strict hidden-state mapping. Categories mapped clearly to the five quantitative hidden states (Code, Test, Dependency, CI/Config, and Something else) are included, yielding 305 usable records out of the original 375.

5.2 Evaluation Cases

The agent was evaluated on 40 cases derived from the original dataset. These 40 cases were held out from the data used to construct the agent's priors and outcome models.

The evaluation therefore separates the historical data used to construct the model from the cases used to test the policy.

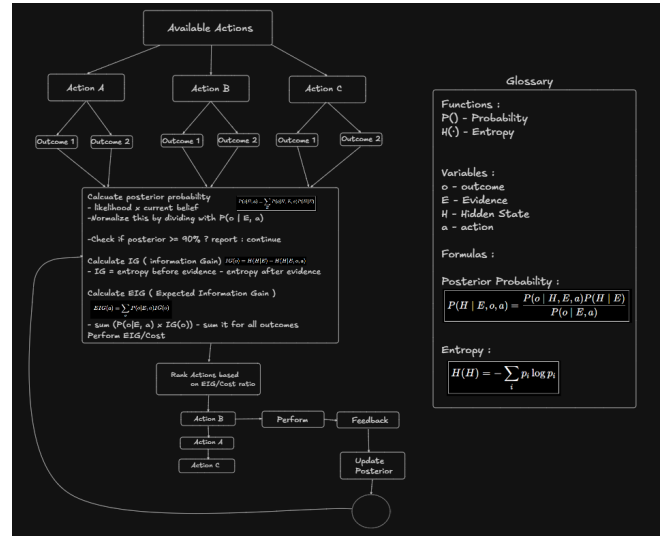


Figure 2: Bayesian and EIG-based action-selection process across the five hidden states. The agent maintains a posterior belief, evaluates possible action outcomes, computes information gain and expected information gain, ranks actions by EIG/cost, observes the selected action's outcome, and updates its posterior.

5.3 Action Outcome Simulation

The original dataset does not directly contain diagnostic-action outcomes. It therefore does not directly provide the probabilities

$$P(o | H, E, a)$$

required for the full EIG calculation.

For V1, these probabilities were estimated using action-specific observable proxies derived from historical step and sub-category fields. The resulting outcome models were used to simulate diagnostic outcomes for the evaluation cases.

This distinction is important: the outcome probabilities are simulation assumptions and are not claimed to be directly observed action probabilities.

5.4 Policies Compared

Three policies were compared.

Baseline. The baseline uses historical probabilities and does not perform diagnostic actions or use a feedback loop.

Cost-only. The cost-only policy ranks available actions by diagnostic cost and performs lower-cost actions first. The motivation is that low-cost actions require less diagnostic effort and may still provide useful information.

EIG/cost. The EIG/cost policy calculates the expected information gain for each available action and ranks actions according to:

$$Score(a) = \frac{EIG(a)}{Cost(a)}.$$

After each action, the observed outcome is incorporated into the posterior distribution and the remaining actions are re-evaluated.

315 5.5 Evaluation Metrics

316 The policies were evaluated using:

- 317 • Accuracy
- 318 • Macro precision
- 319 • Macro recall
- 320 • Human-review rate
- 321 • Average diagnostic cost
- 322 • Average number of actions
- 323 • Report precision

324 6 Results

325 The EIG/cost policy achieved the highest accuracy, macro
326 precision, macro recall, and report precision among the three
327 evaluated policies.

328 Compared with the cost-only policy, EIG/cost improved
329 accuracy by 5 percentage points and macro recall by 5 per-
330 centage points. Macro precision increased by 2.17 percent-
331 age points. The human-review rate decreased by 2.5 percent-
332 age points. The average diagnostic cost decreased from 6.93
333 to 6.46, while the average number of actions decreased from
334 5.23 to 4.68.

335 The baseline had zero diagnostic cost and zero actions be-
336 cause it does not perform diagnostic actions. However, its
337 accuracy and precision were substantially lower than those of
338 the two sequential diagnostic policies.

339 The results suggest that considering expected information
340 gain together with diagnostic cost can improve the trade-off
341 between diagnostic effort and root-cause identification under
342 the constructed simulation environment.

343 7 Failure Analysis

344 Five failure cases were recorded across three major patterns.

345 7.1 Confident but Incorrect Diagnosis

346 In Case 1, the agent became confident in Dependency even
347 though the true root cause was CI/Config. This is an expen-
348 sive type of failure because the agent performed diagnostic
349 actions and eventually produced a confident but incorrect re-
350 port.

351 The research attributes this behavior in V1 partly to the
352 construction of the outcome models. The original dataset
353 does not contain direct information about action outcomes
354 conditioned on the hidden root cause. The action outcome
355 models were therefore simulated using manually mapped
356 proxies. An incorrect or overly strong simulated outcome can
357 push the posterior towards the wrong hidden state.

358 7.2 Misleading Evidence

359 Cases 3, 4, and 10 illustrate failures in which early evidence
360 shifted the posterior toward an incorrect hidden state. Later
361 evidence moved the posterior in the correct direction, but the
362 probability assigned to the incorrect state had already become
363 sufficiently large that the true root cause could not recover
364 enough probability to reach the reporting threshold.

This behavior highlights a limitation of the current out-
come model. There were not enough directly observed
evidence-outcome relationships to ensure that every simu-
lated piece of evidence would provide a realistic signal for
the correct hidden state.

370 7.3 Correct Direction but Insufficient Confidence

371 Case 9 reached 88.3% probability for CI/Config but remained
372 below the 90% reporting threshold. In this situation, the
373 diagnostic evidence was moving the belief towards the cor-
374 rect root cause, but the static stopping criterion prevented the
375 agent from reporting it.

376 The 90% threshold was selected heuristically rather than
377 estimated from historical decision outcomes. A larger dataset
378 containing real diagnosis outcomes could be used to calibrate
379 this threshold.

380 7.4 Code Recall

381 Both EIG/cost and cost-only achieved 0% recall for Code on
382 the evaluation set. This indicates that the current diagnostic
383 outcome models did not adequately distinguish Code failures
384 from the other root causes in this evaluation.

385 Overall, the failures can be summarized into three patterns:

- 386 1. Confident but incorrect diagnosis.
- 387 2. Misleading evidence suppressing the true root cause.
- 388 3. Correct evidence but insufficient confidence, resulting in
389 escalation.

390 8 Limitations, Ethics, and Human Control

391 The main limitation of the V1 system is the lack of di-
392 rectly observed diagnostic-action outcomes in the dataset.
393 The dataset contains labeled CI failures but does not directly
394 record the outcomes that would have been obtained by per-
395 forming actions such as inspecting dependencies, compar-
396 ing with a previous successful run, or inspecting server state.
397 Consequently, the probabilities

$$P(o \mid H, E, a)$$

398 were estimated using action-specific proxies. These probabil-
399 ities are used for simulation and should not be interpreted as
400 directly observed action probabilities.

401 A second limitation is the size of the evaluation. Only
402 40 held-out cases were used. The resulting metrics there-
403 fore have substantial sampling uncertainty and should not be
404 treated as statistically conclusive evidence of production per-
405 formance.

406 A third limitation is the heuristic reporting threshold. The
407 static 90% threshold was selected using intuition rather than
408 being calibrated from historical decision outcomes.

409 The system is designed to keep a human in the decision
410 loop. When the agent cannot reach sufficient confidence after
411 the useful available diagnostic actions have been exhausted,
412 it escalates the case rather than forcing a root-cause report.
413 This is particularly important because a confident but incor-
414 rect report can waste diagnostic effort and potentially mislead
415 the human reviewer.

Table 3: Comparison of the three diagnostic policies over 40 held-out evaluation cases.

Metric	EIG/cost	Cost-only	Baseline
Accuracy	55%	50%	25%
Macro Precision	65.23%	63.06%	6.25%
Macro Recall	55%	50%	25%
Human-review rate	35%	37.5%	0%
Average diagnostic cost	6.46	6.93	0
Average actions	4.68	5.23	0
Report precision	84.6%	80%	25%

The probability record also provides an audit trail of the agent’s reasoning. It records evidence, hidden states, beliefs, actions, likelihoods, posterior probabilities, outcomes, decisions, and version information for an individual case.

The agent is therefore intended as a diagnostic aid rather than a replacement for human engineering judgment.

9 Conclusion and Future Scope

This work presents a CI diagnosis agent that treats the root cause of a CI failure as a hidden state spanning five distinct categories and uses Bayesian belief updates to reason about possible causes. Instead of following a fixed debugging sequence, the agent selects diagnostic actions using expected information gain relative to diagnostic cost. The result of each action is fed back into the belief engine, allowing the agent to update its beliefs and select another action.

The V1 evaluation compared EIG/cost, cost-only, and a historical-probability baseline over 40 held-out cases. EIG/cost achieved 55% accuracy, 65.23% macro precision, 55% macro recall, and 84.6% report precision. It also required fewer actions and lower average diagnostic cost than the cost-only policy.

At the same time, the failure analysis shows that the current system is limited by the quality of the simulated action outcome models and the static decision rule. To address these limitations, future iterations will explore the following architectural extensions:

9.1 Jensen-Shannon Divergence Policy

While Shannon Entropy and EIG measure total uncertainty reduction, they do not directly evaluate how effectively an action differentiates between specific competing hypotheses. Future scope includes replacing or augmenting the EIG/cost policy with a selection metric based on Jensen-Shannon Divergence (JSD). For a set of candidate posterior belief distributions $P_o = P(H | E, o, a)$ resulting from possible outcomes $o \in \mathcal{O}$, the JSD is defined as:

$$\text{JSD}(P_{o_1}, P_{o_2}, \dots) = H \left(\sum_o w_o P_o \right) - \sum_o w_o H(P_o)$$

where $w_o = P(o | E, a)$. Selecting actions that maximize $\frac{\text{JSD}(a)}{\text{Cost}(a)}$ ensures that the chosen diagnostic step maximizes the statistical distance between alternative state hypotheses, preventing the agent from selecting low-cost actions that provide information without resolving ambiguity between top candidate causes.

Table 4: Proposed state-specific stopping thresholds p_i^* based on domain risk.

Hidden State (H_i)	C_{FP}	C_{FN}	Threshold (p_i^*)
Code	\$200	\$450	30.8%
Test	\$50	\$450	10.0%
Dependency	\$10	\$450	2.2%
CI/Config	\$10	\$450	2.2%
Something else	\$100	\$450	18.2%

9.2 State-Specific Optimal Stopping Thresholds

Rather than relying on an arbitrary constant threshold (e.g., 90%), future work will implement state-specific stopping thresholds derived from formal Bayesian decision theory. By defining a cost matrix representing the cost of a false positive ($C_{FP,i}$) and false negative ($C_{FN,i}$) for each hidden state H_i , the optimal decision threshold p_i^* can be formally derived as:

$$p_i^* = \frac{C_{FP,i}}{C_{FP,i} + C_{FN,i}}$$

Table 4 outlines proposed state-specific thresholds based on domain risk profiles.

Under this framework, the agent will stop diagnostic actions and report root cause H_i as soon as its posterior belief satisfies $P(H_i | E) \geq p_i^*$, balancing risk dynamics far more effectively across high-risk changes (e.g., Code) versus lower-risk fixes (e.g., Dependency).

Ethical Statement

The agent is designed to support diagnosis rather than replace human decision-making. Its probabilistic output can be incorrect, and the evaluation demonstrates several failure modes in which the agent either reports an incorrect root cause or cannot reach the required confidence. For this reason, the system includes human escalation when sufficient confidence is not achieved.

The main ethical concern in the current system is overconfidence. A high probability does not guarantee that the diagnosis is correct, particularly because the V1 action outcome probabilities are simulated rather than directly observed. The system should therefore not be treated as an autonomous authority for production incident decisions without further validation.

Contribution Statement

Problem selection: Panduru Grishm Siddharth.

Public discussions: Panduru Grishm Siddharth.

Agent design: Panduru Grishm Siddharth.

Software development: Panduru Grishm Siddharth.

Tests: Panduru Grishm Siddharth.

Citation checks: Panduru Grishm Siddharth.

Preprint sections: Panduru Grishm Siddharth.

PDF checks: Panduru Grishm Siddharth.

AI-Use Statement

AI tools were used during the project for coding and debugging assistance, mathematical explanations, research organization, editing and writing assistance, and data analysis. The resulting code, analysis, decisions, and written material were reviewed by the author.

Code and Project Materials

The code and project materials are available at: <https://github.com/Proudprogamer/CI-Agent>